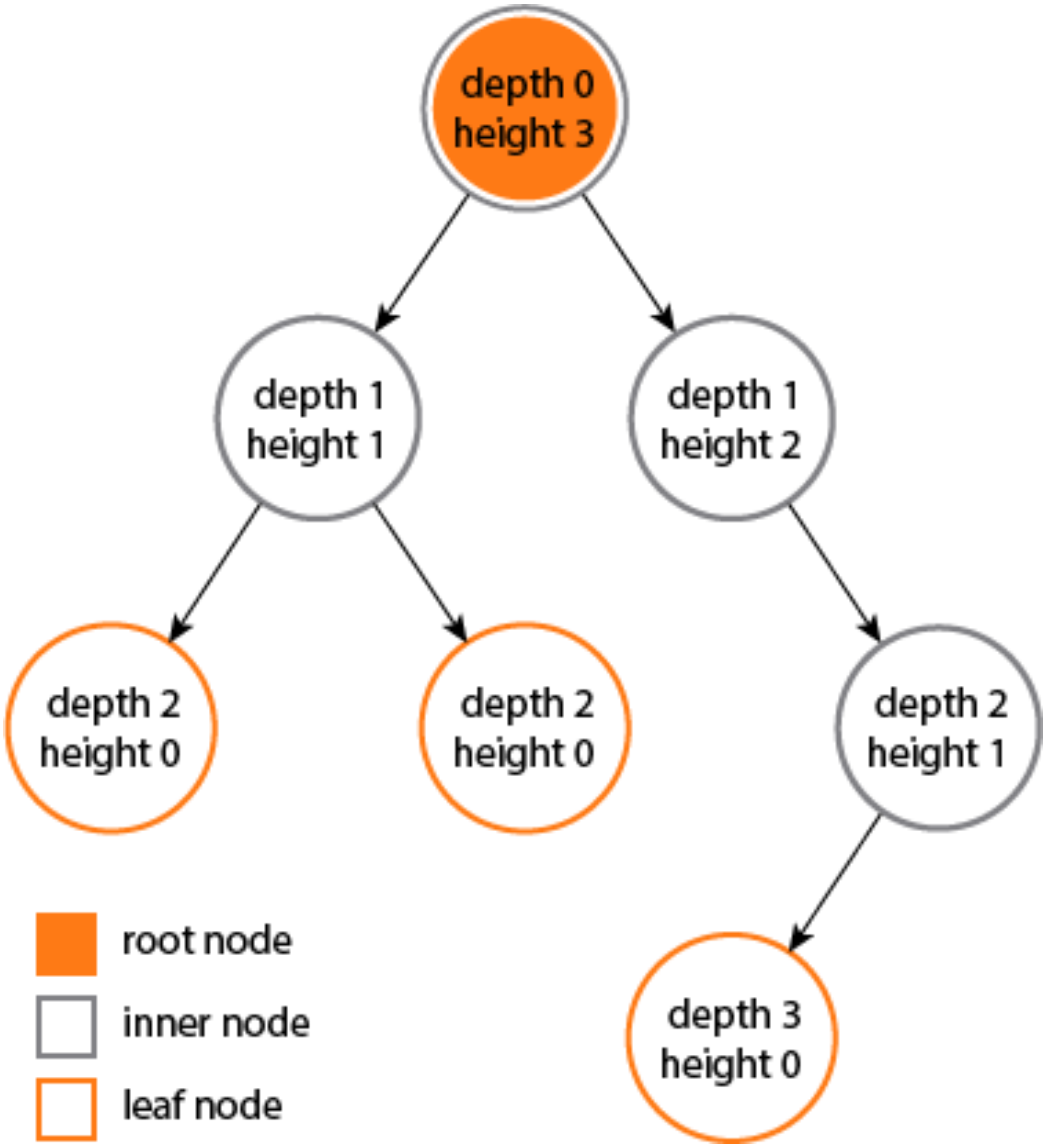
Tree

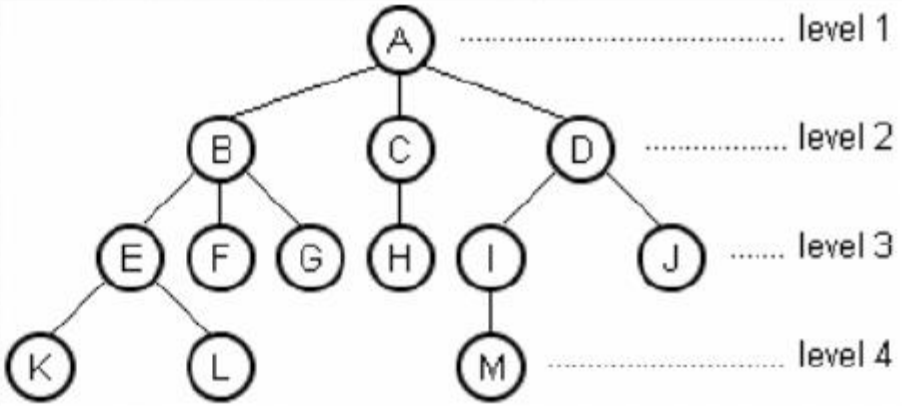
Algoritma & Struktur Data

Tree

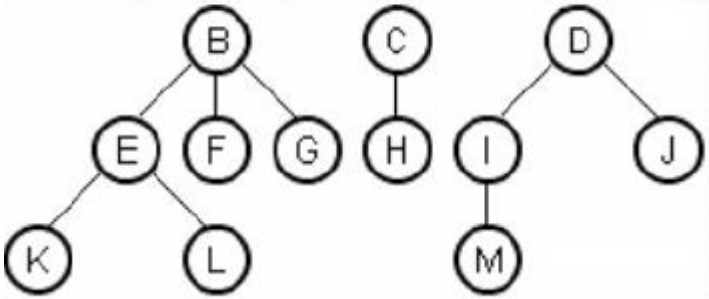
- Tree adalah suatu struktur data (juga tipe data abstrak) yang terdiri dari sejumlah simpul (**node**) yang
  - Terdapat satu simpul khusus yang disebut **root**
  - Simpul lainnya dibagi ke dalam sejumlah himpunan terpisah T1, T2, …, Tn. Masing-masing himpunan ini merupakan sebuah tree
  - T1, T2, …, Tn disebut **subtree** dari root


depth 0
height 3
depth 1
height 1
depth 1
height 2
depth 2
height 0
depth 2
height 0
depth 2
height 1
depth 3
height 0
root node
inner node
leaf node

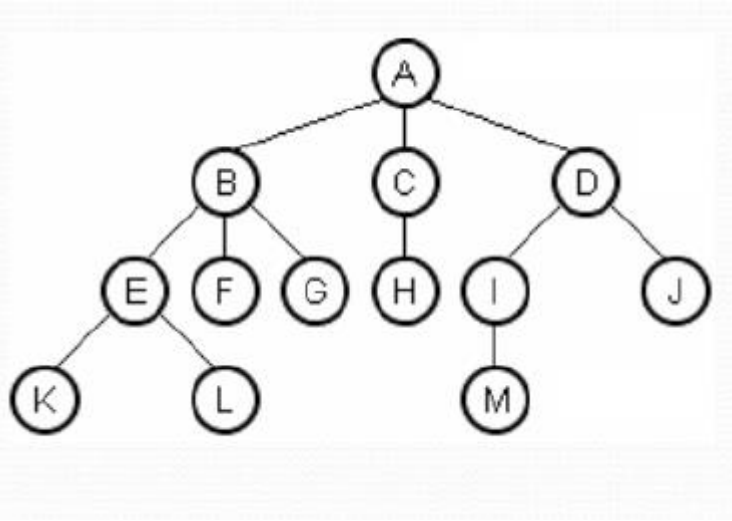
Simpul A ⇨ root.
Height ⇨ 4
Depth ⇨ 4



Subtree dari A



Height M ? 0
Depth J ? 1
Height E ? 1
Height K ? 0
Depth L ? 2

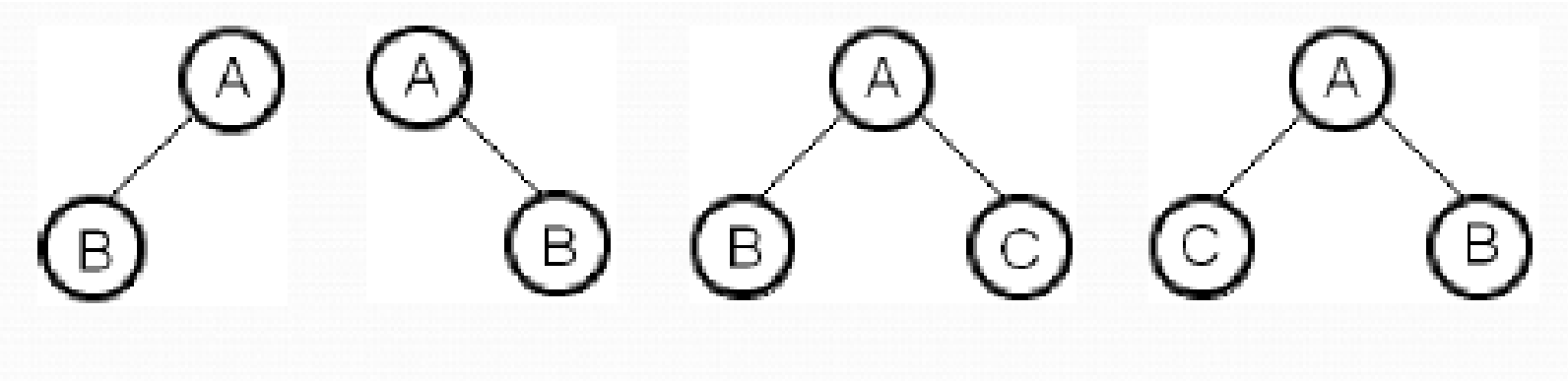


A parent dari ??	B, C, dan D
C parent dari ??	H
B children dari ??	A
M children dari ??	I
E,F, dan G adalah ?	sibling

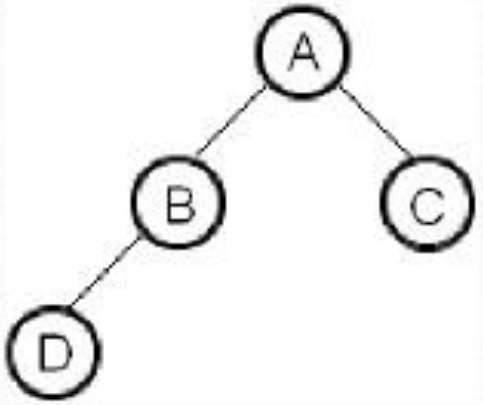
Ancestor dari G ??	A dan B
Ancestor dari M ??	A, D, dan I
Descendant dari B ??	E, F, G, K, dan L

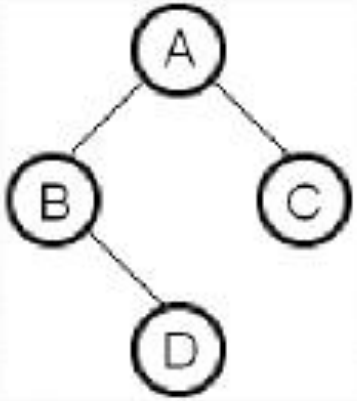
Binary Tree

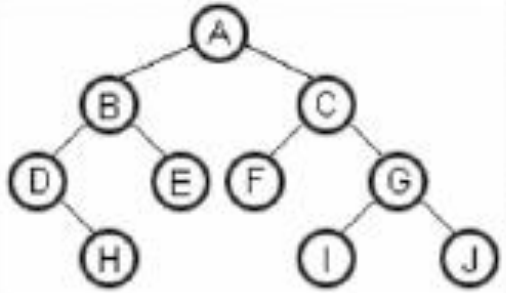
- Adalah sejenis tree yang setiap simpulnya mempunyai paling banyak dua subtree
- Kedua subtree itu disebut **left subtree** dan **right subtree**



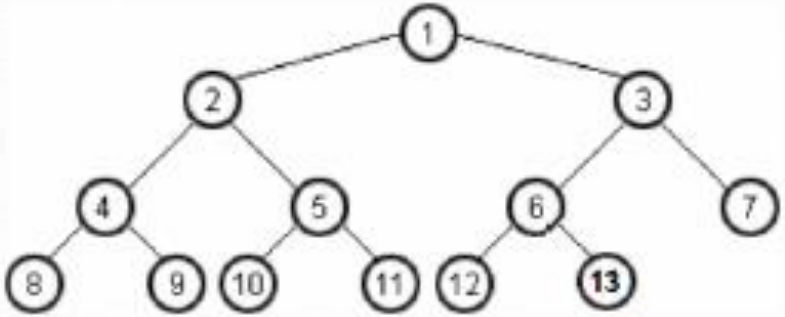
Binary Tree





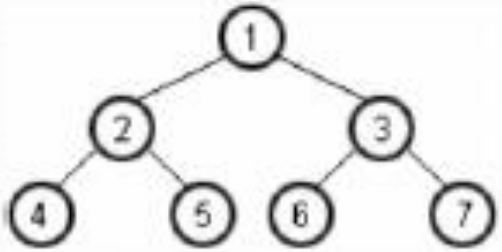


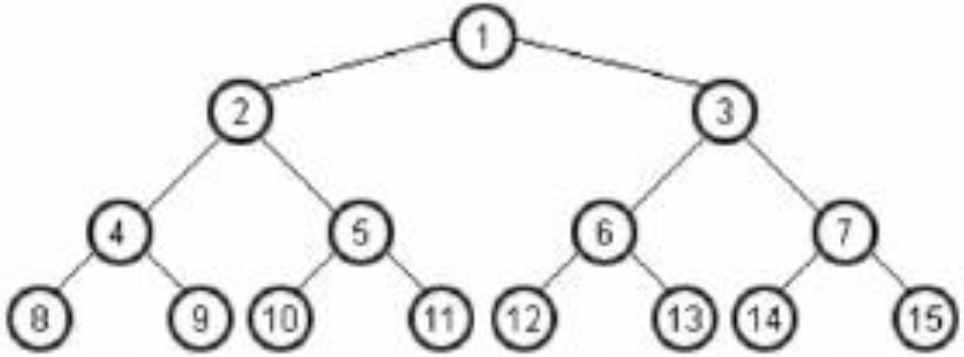
Complete binary tree



Binary Tree

Full binary tree

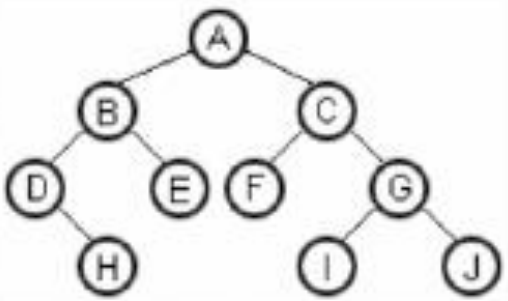


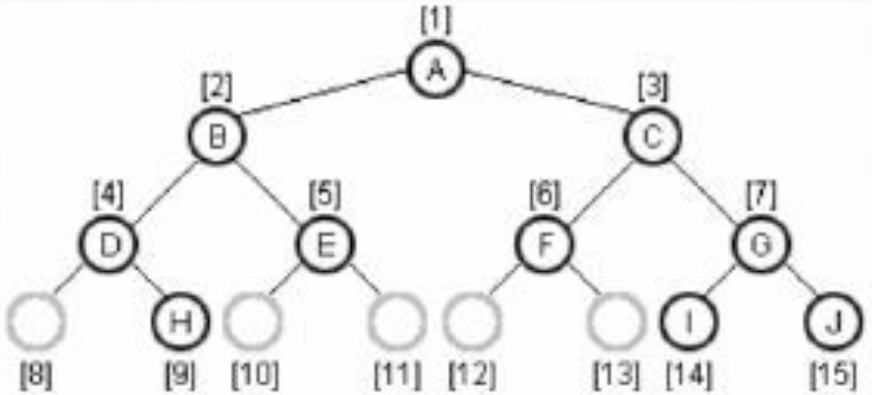


Jumlah maksimum simpul pada level i adalah $2^i - 1$.
Jumlah maksimum simpul pada binary tree dengan height h adalah $2^h - 1$.

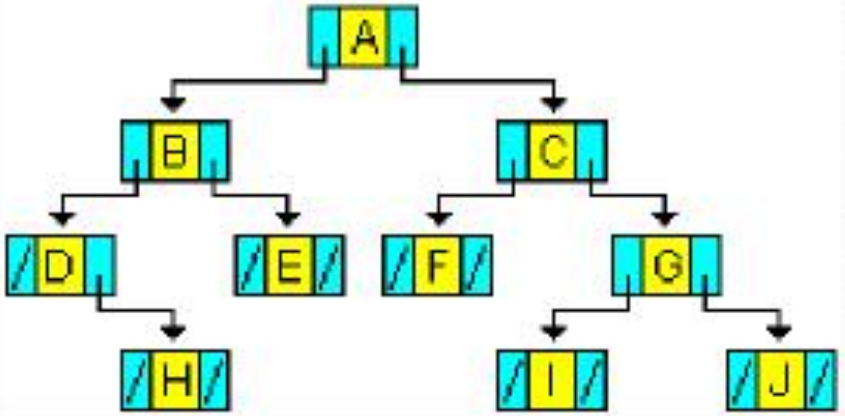
Representasi

Representasi Array





Representasi Linked



Operasi

Operasi terhadap *binary tree*

1. **creattree**, membentuk binary tree kosong tanpa simpul
2. **isemptytree**, memeriksa apakah sebuah binary tree kosong
3. **insert**, menambah data (simpul) ke dalam binary tree
4. **traversal**, menelusuri simpul-simpul pada binary tree
5. **deletenode**, menghapus simpul

Tree dengan Representasi Linked List

```
struct tbintree{
  int data;
  struct tbintree *left, *right;
} ;

void createtree(struct tbintree **root) {
  *root = NULL;
}
int isemptytree(struct tbintree *root) {
  if (root == NULL) return 1;
  return 0;
}
```

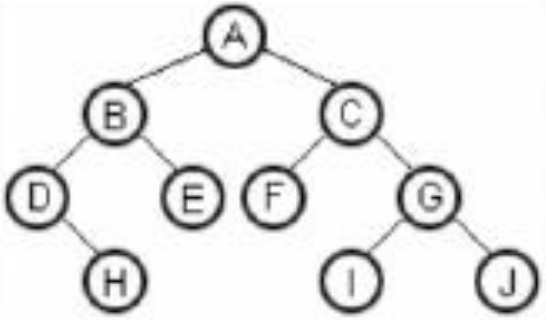
Tree dengan Representasi Linked List

```
void insert(struct tbintree **root, int data, char posisi) {
  struct tbintree *node;

  node = (struct tbintree*) malloc (sizeof(struct tbintree));
  node->data = data;
  node->left = node->right = NULL;

  if (isemptytree(*root))
     *root = node;
  else
     switch (posisi) {
        case 'L': if ((*root)->left == NULL)
                     (*root)->left = node; break;
        case 'R': if ((*root)->right == NULL)
                     (*root)->right = node;
     }
}
```

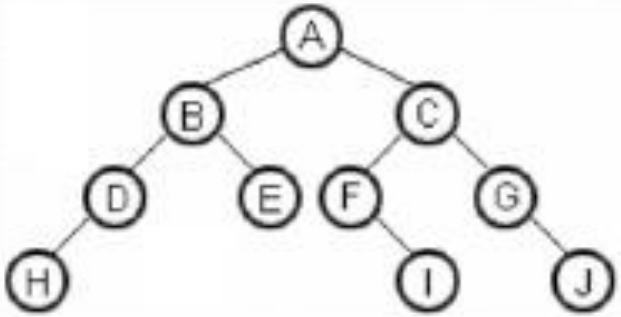
Traversal

- Jika disimbolkan:
  - L untuk subtree kiri
  - R untuk subtree kanan
  - P untuk proses terhadap simpul induk
  - Maka terdapat 6 jenis traversal:
    - PLR, LPR, LRP,
    - PRL, RPL, RLP
- Tiga jenis yang sering digunakan:
  - PLR, disebut **preorder** : induk, kiri, kanan
  - LPR, disebut **inorder** : kiri, induk, kanan
  - LRP, disebut **postorder** : kiri, kanan, induk

Traversal



preorder: A B D H E C F G I J
inorder: D H B E A F C I G J
postorder: H D E B F I J G C A



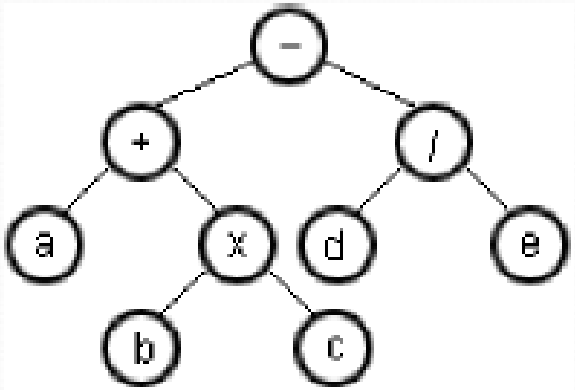
preorder: A B D H E C F I G J
inorder: H D B E A F I C G J
postorder: H D E B I F J G C A

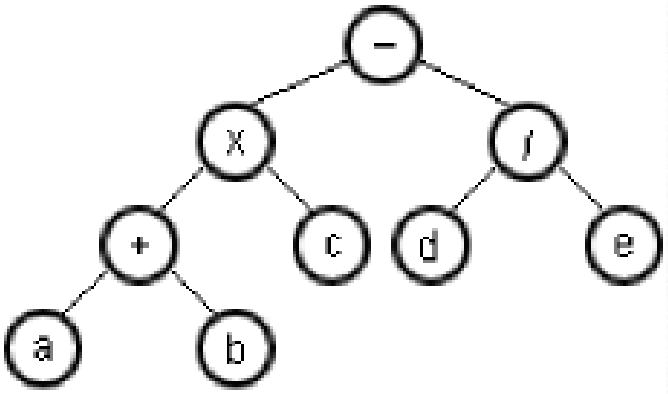
Expression Tree

Expression tree adalah binary tree yang menggambarkan suatu ekspresi

a + b x c – d / e

Preorder: - + a x b c / d e
Inorder: a + b x c – d / e
Postorder: a b c x + d e / -





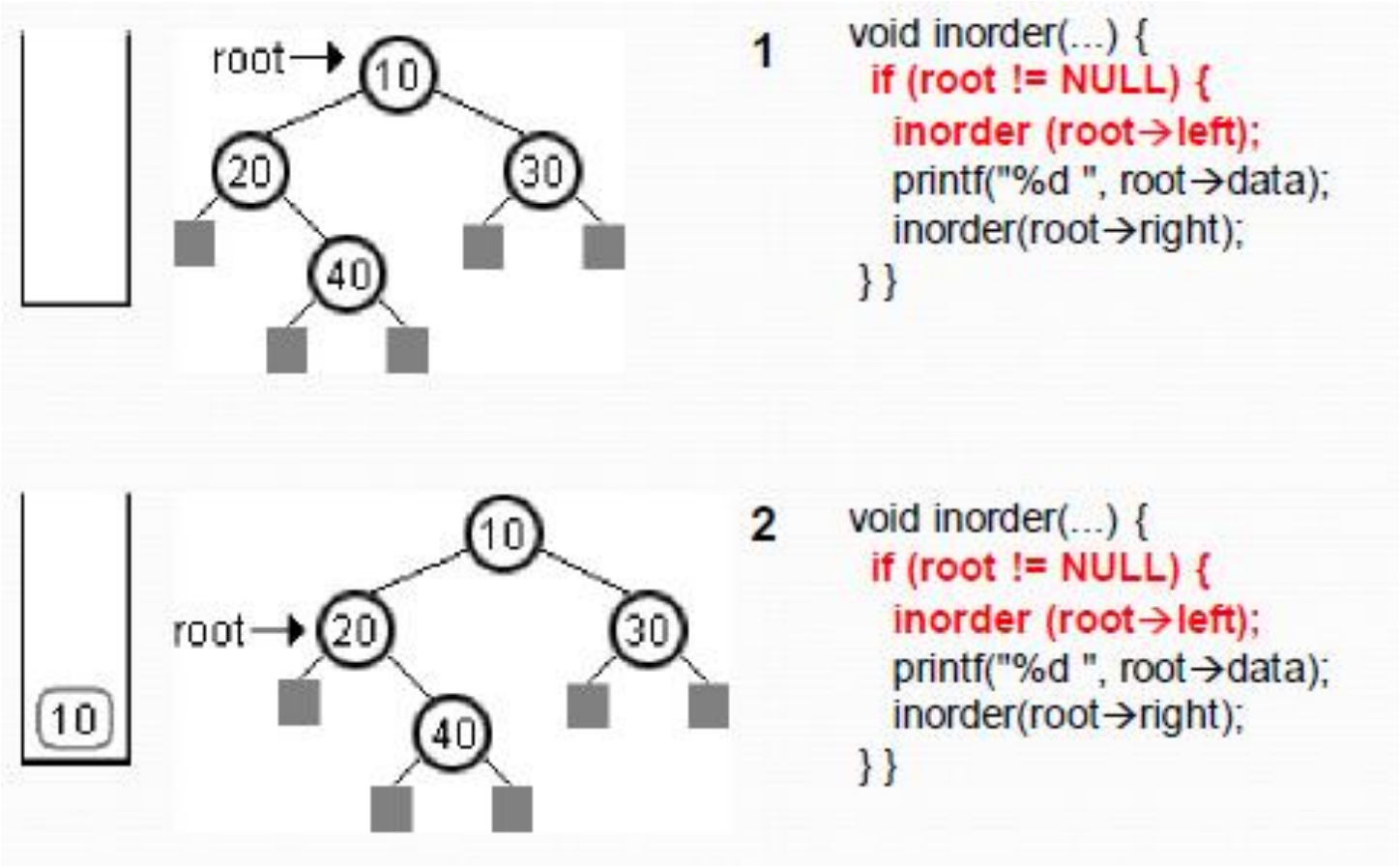
(a + b) x c – (d / e)

Preorder: - x + a b c / d e
Inorder: a + b x c – d / e
Postorder: a b + c x d e / -

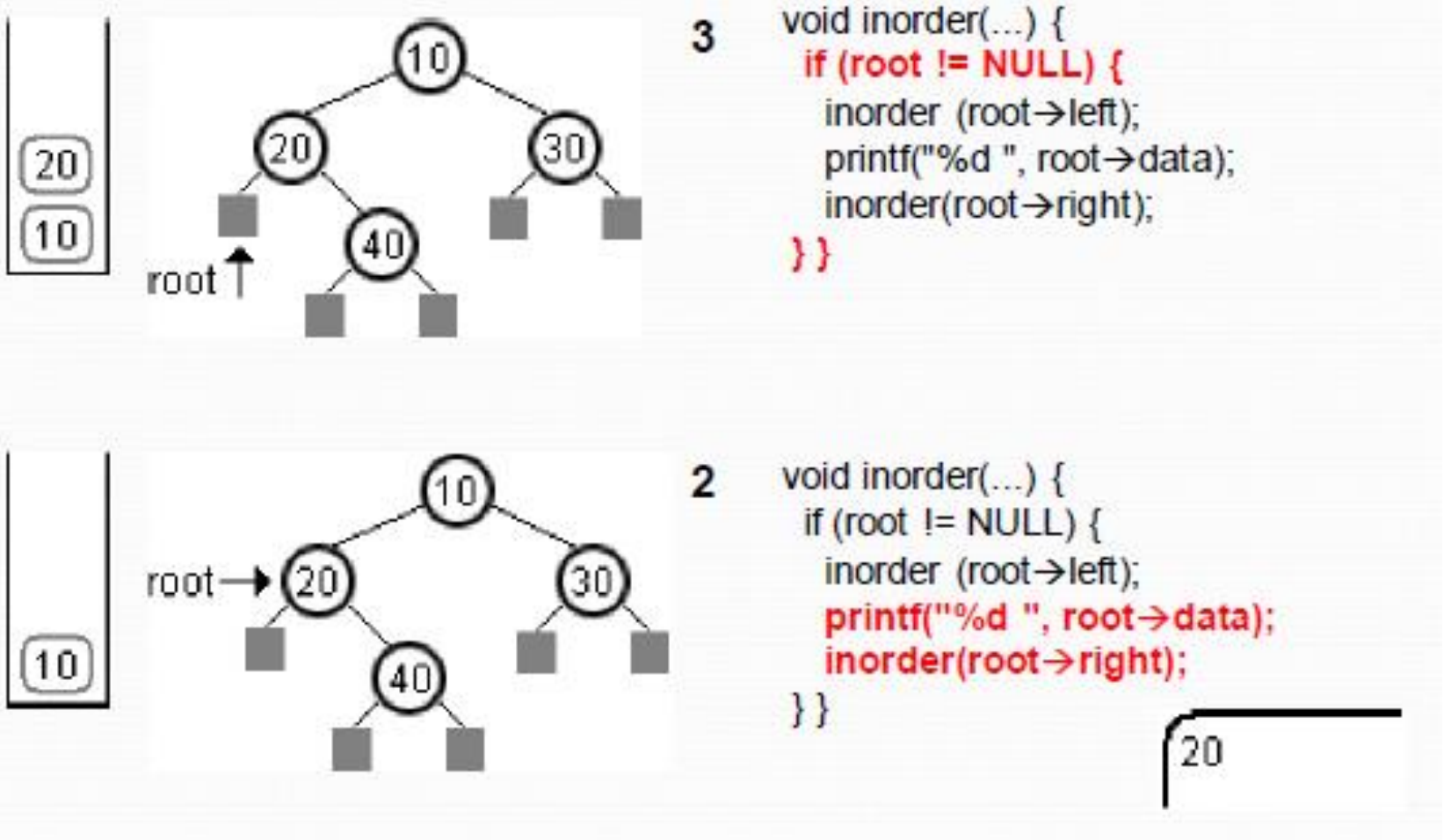
Program

```
void preorder(struct tbintree *root) {
  if (root != NULL) {
    printf("%d ", root->data);
    preorder (root->left);
    preorder(root->right);
  }
}
void inorder(struct tbintree *root) {
  if (root != NULL) {
    inorder (root->left);
    printf("%d ", root->data);
    inorder(root->right);
  }
}
void postorder(struct tbintree *root) {
  if (root != NULL) {
    postorder (root->left);
    postorder(root->right);
    printf("%d ", root->data);
  }
}
```

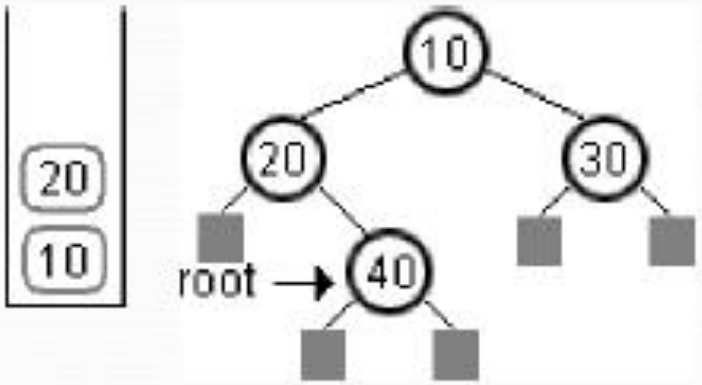
Recursive Inorder



Recursive Inorder

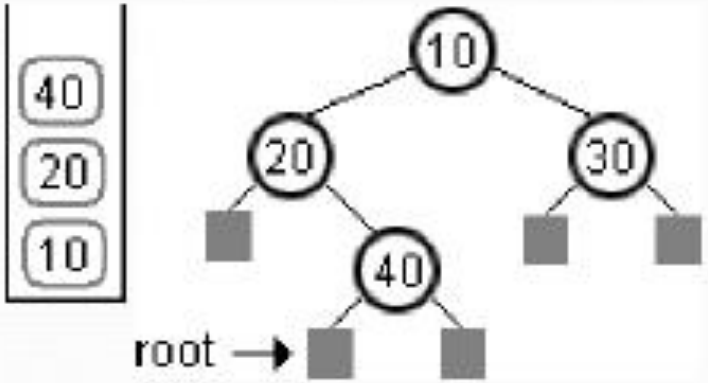


Recursive Inorder



3

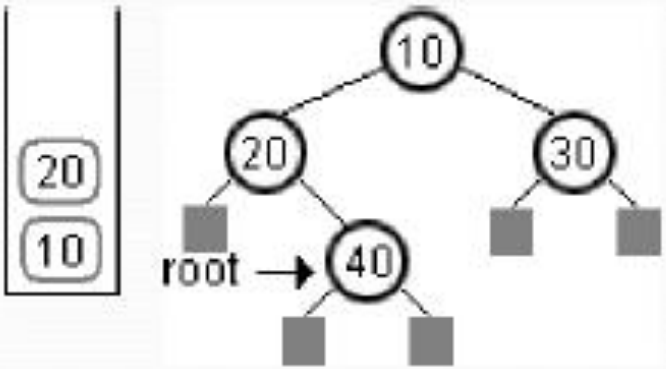
```
void inorder(...) {
 if (root != NULL) {
   inorder (root→left);
   printf("%d ", root→data);
   inorder(root→right);
}}
```



4

```
void inorder(...) {
 if (root != NULL) {
   inorder (root→left);
   printf("%d ", root→data);
   inorder(root→right);
}}
```

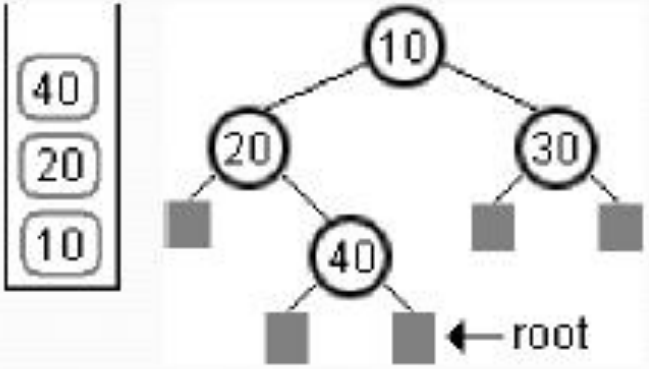
Recursive Inorder



3

```
void inorder(...) {
  if (root != NULL) {
    inorder (root→left);
    printf("%d ", root→data);
    inorder(root→right);
} }
```

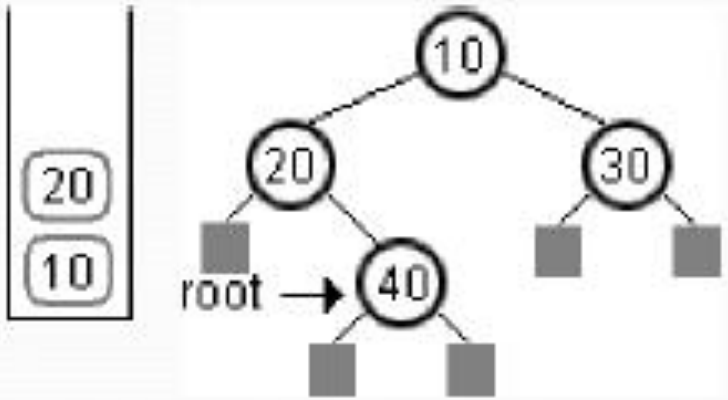
20 40



4

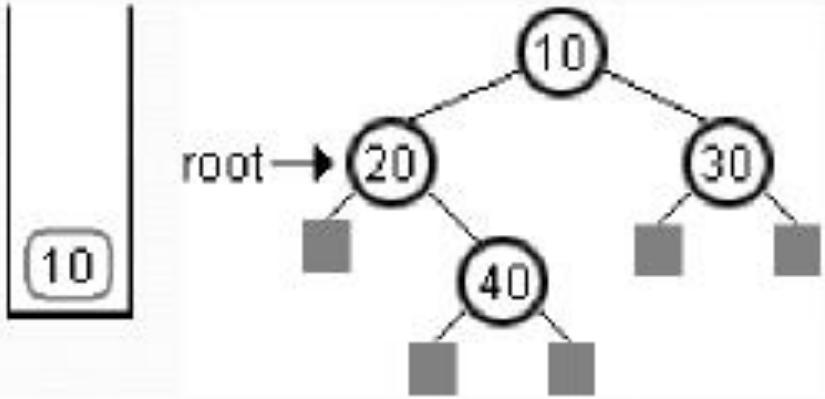
```
void inorder(...) {
  if (root != NULL) {
    inorder (root→left);
    printf("%d ", root→data);
    inorder(root→right);
} }
```

Recursive Inorder



3

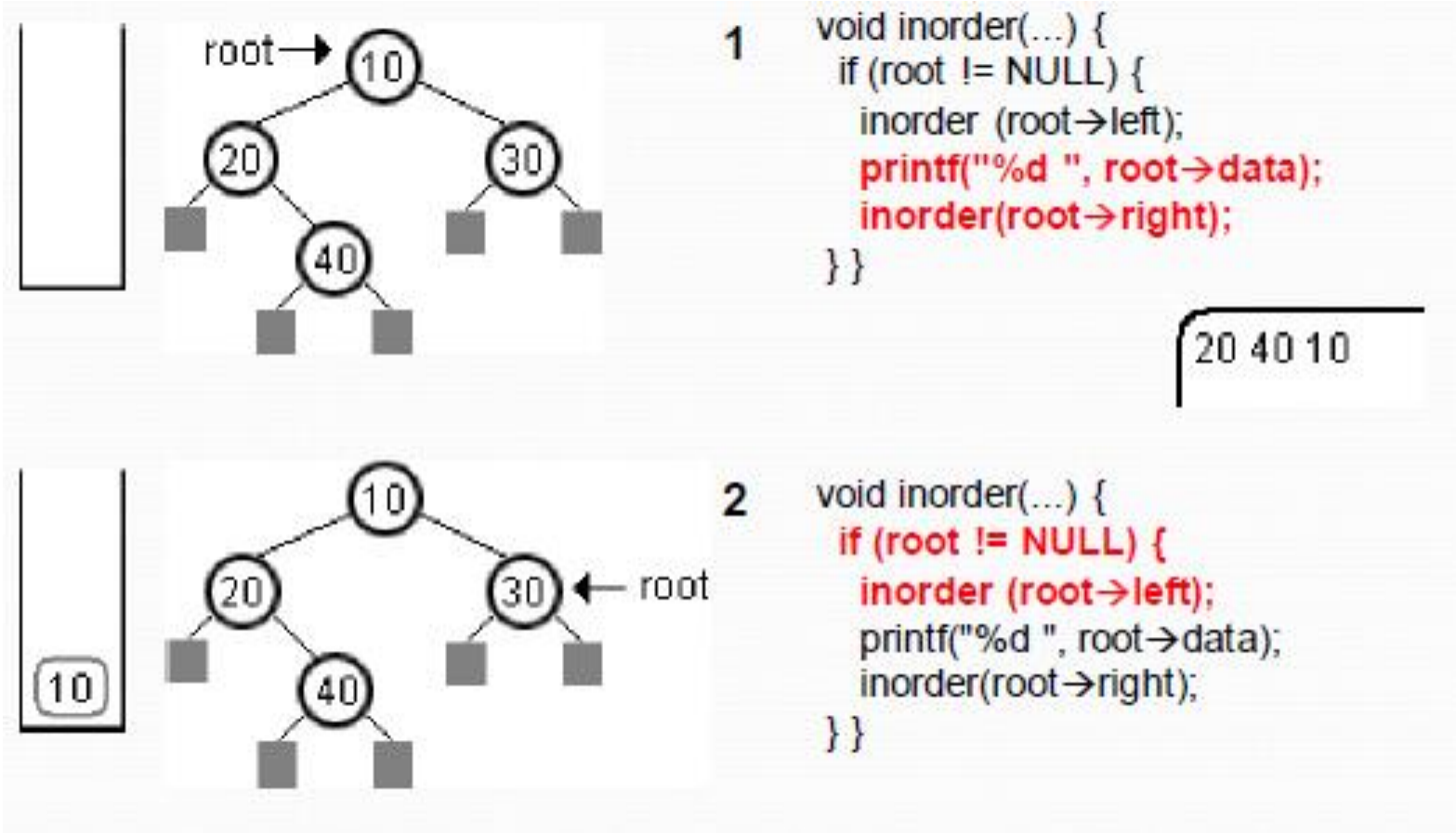
```
void inorder(...) {
  if (root != NULL) {
    inorder (root→left);
    printf("%d ", root→data);
    inorder(root→right);
}}
```



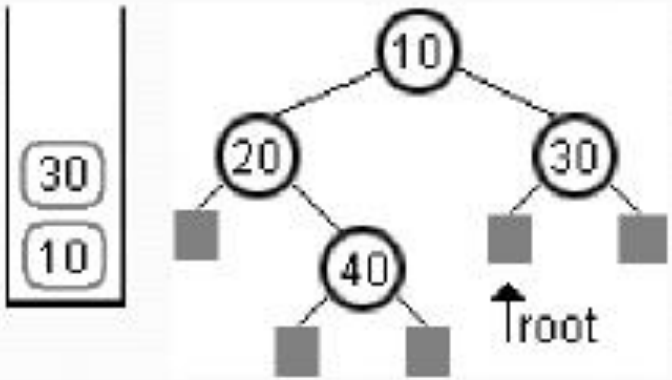
2

```
void inorder(...) {
  if (root != NULL) {
    inorder (root→left);
    printf("%d ", root→data);
    inorder(root→right);
}}
```

Recursive Inorder

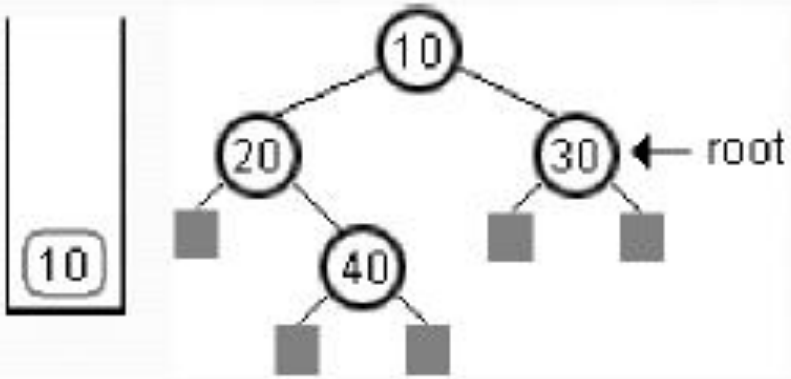


Recursive Inorder



3

```
void inorder(...) {
 if (root != NULL) {
   inorder (root→left);
   printf("%d ", root→data);
   inorder(root→right);
}}
```

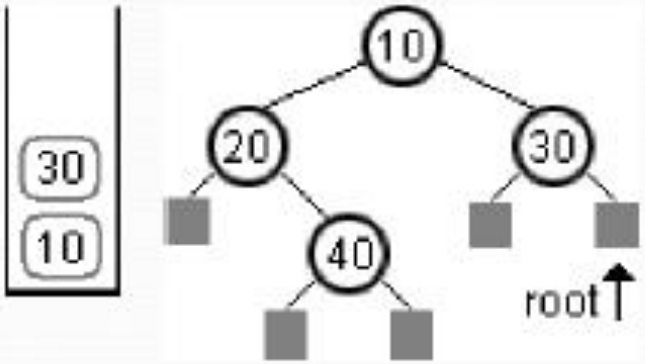


2

```
void inorder(...) {
 if (root != NULL) {
   inorder (root→left);
   printf("%d ", root→data);
   inorder(root→right);
}}
```

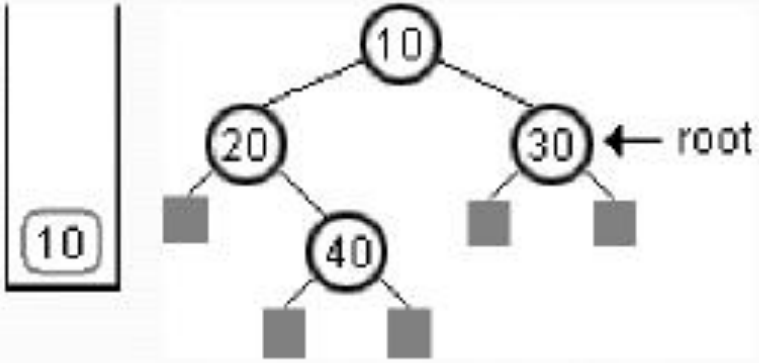
20 40 10 30

Recursive Inorder



3

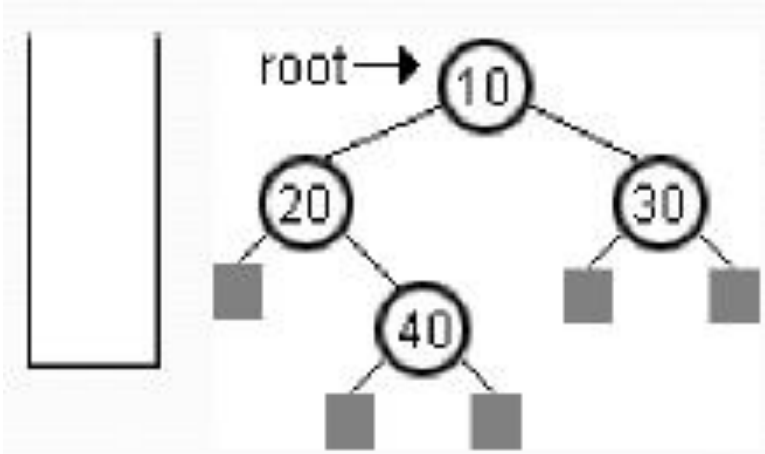
```
void inorder(...) {
  if (root != NULL) {
    inorder (root→left);
    printf("%d ", root→data);
    inorder(root→right);
  }}
```



2

```
void inorder(...) {
  if (root != NULL) {
    inorder (root→left);
    printf("%d ", root→data);
    inorder(root→right);
  }}
```

Recursive Inorder



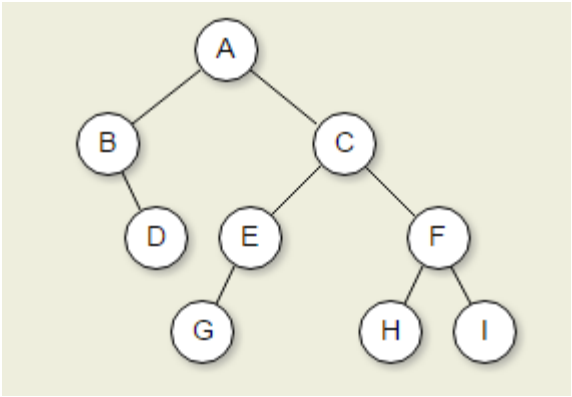
1

```
void inorder(...) {
  if (root != NULL) {
    inorder (root→left);
    printf("%d ", root→data);
    inorder(root→right);
} }
```

Reference

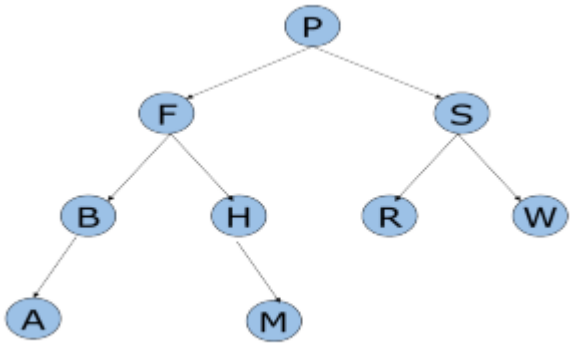
1. Ngoen, T.S., (2004), *Data Structures*, IBII Lecture Notes
2. Mark Allen Weiss, 1997, *Data Structures and Algorithm Analysis in C*, Addison Wesley Longman, inc.
3. Horowitz,E., Sahni,S. and Anderson-Freed,S.,(1993), *Fundamentals of Data Structures in C*, Computer Science Press,New York.

Latihan 1



- Diberikan sebuah tree sebagai berikut. Lakukan traversal secara:
  - Preorder
  - Inorder
  - Postorder

Latihan 2



- Diberikan tree sebagai berikut. Lakukan traversal secara:
  - Preorder
  - Inorder
  - Postorder

Latihan 3

- Buatlah sebuah binary tree dari hasil traversal sebagai berikut:

preorder: A B D H E C F I G J
inorder: H D B E A F I C G J
postorder: H D E B I F J G C A